

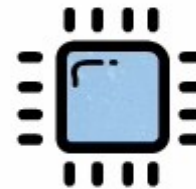
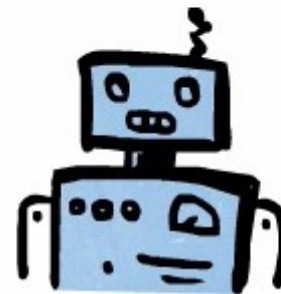
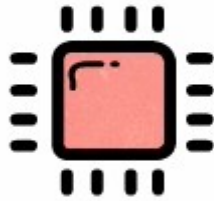
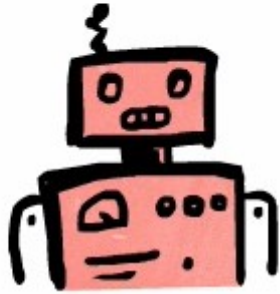
PARALEL HESAPLAMA

Öğr. Gör. Zafer SERİN

Processler Arası Bellek Paylaşımı

- Process'ler oluşturulduğu zaman bellekte kendilerine ait ayrı bir bellek alanı oluşturulur ve bu alanda ilgili Process çalışır.
- Thread'ler zaten ortak bir bellek alanında çalıştığı için böyle bir yapıya ihtiyaç duyulmaz.

Processler Arası Bellek Paylaşımı



Process ve Bellek İzolasyonu

- Bilindiği üzere, her Process işletim sisteminde kendine ait izole bir bellek alanına sahiptir. Bir process içinde tanımlanan değişken (örneğin bir liste veya sayaç), diğer process tarafından doğrudan görülemez. Thread'lerin aksine, Process'ler belleği paylaşmaz. Bu durum veri güvenliği (safety) sağlar ancak iletişimi zorlaştırır. Bu iletişim zorluğununun aşılması için IPC (Inter-Process Communication) araçlarını kullanırız.

Neden İletişim Kurmalıyız?

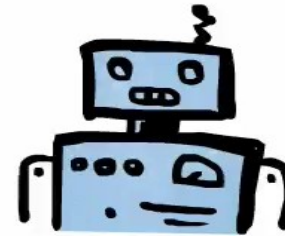
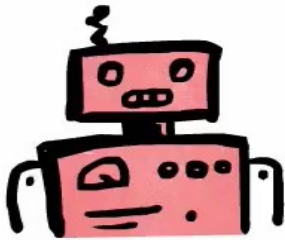
- Sadece veri göndermek için değil, iş akışını yönetmek için de iletişim şarttır.
- 1. İş Dağıtımı: Bir process veriyi internetten çeker (Producer), diğeri veriyi temizler (Worker), bir diğeri veritabanına yazar (Consumer).
- 2. Senkronizasyon: Bir işin bittiğini diğerine haber vermek için.
- 3. Durum Paylaşımı: Uygulamanın sağlık durumunu veya hataları ana sürece bildirmek için.

Message Passing (Mesajlaşma) yöntemi, bu senaryolarda process'lerin birbirine veri paketleri göndererek anlaşmasını

Message Passing Kullanarak İletişim

- Message Passing yöntemi ile iletişim kurulması tıpkı insanların kullandığı iletişim yöntemine benzer.
- İnsanlar iletişim kurarken birbirlerine mesaj veya e-mail atarak hatta konuşarak iletişim kurarlar. Bu durumda arkadaşınızla iletişim kurarken ona bir şey söylersiniz, arkadaşınız bu söylediğinizi anlar ve buna karşılık size bir şey söyler böylece iletişim devam eder.

Message Passing Kullanarak İletişim



TOOLS

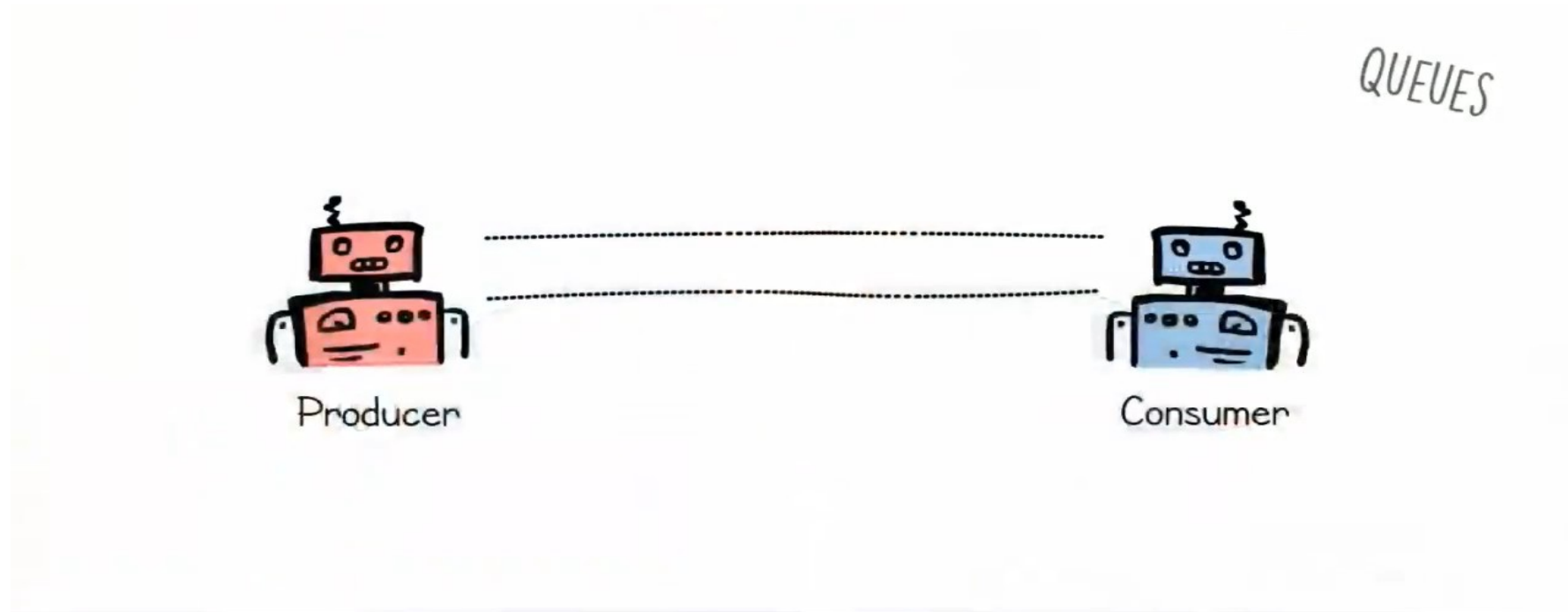
Queue (Kuyruk) Yapısının Temelleri

- multiprocessing.Queue, Python'daki queue.Queue yapısının process tabanlı versiyonudur. Thread-safe ve Process-safe bir yapıdır; yani aynı anda birden fazla Process kuyruğa veri yazmaya veya okumaya çalışsa bile veri bozulmaz. Kuyruk, FIFO (First-In, First-Out) mantığıyla çalışır; ilk giren veri ilk çıkar.

Queue (Kuyruk) Yapısının Temelleri

- Bir baş(front) ve son(rear) içerir.
- Queue kapasitesi elle belirlenebilir.
- Bir Producer(Sağlayıcı - Gönderici) put() ile kuyruğa bir mesaj ekler ve eklenen bu mesaj Consumer(Alıcı) tarafından get() ile okunur.
- Birden çok Producer ve birden çok Consumer olabilir.

Queues(Kuyruklar) ile İletişim



Queue (Kuyruk) Yapısının Temelleri

```
from queue import Queue
from threading import Thread
import time

def consumer(q):
    while True:
        txt = q.get()
        print(txt)
        time.sleep(1)

def producer(q):
    while True:
        q.put("Merhaba")
        print("Mesaj gönderildi")
        time.sleep(1)

q = Queue()
t1 = Thread(target=consumer, args=(q, ))
t2 = Thread(target=producer, args=(q, ))
t1.start()
t2.start()
```

Blocking Kavramı

- Kuyruklar varsayılan olarak Blocking modda çalışır.
- get() Bloklamaları: Eğer bir Consumer get() çağırırsa ve kuyruk o an boşsa, process orada durur (suspend). Veri gelene kadar bir sonraki satıra geçmez.
- put() Bloklamaları: Eğer kuyruğa bir kapasite verilmişse ve kuyruk doluysa, Producer put() yapmaya çalıştığında yer açılana kadar bekler. Bu davranış CPU'yu yormadan beklemeyi sağlar, ancak programın donmuş gibi görünmesine de neden olabilir.

Bloklama Yönetimi ve timeout Parametresi

- Process'in sonsuza kadar beklemesini önlemek için timeout parametresi kullanılır. `q.get(timeout=5)` komutu, "Veri için 5 saniye bekle, gelmezse hata ver" anlamına gelir. Eğer süre dolarsa `queue.Empty` hatası fırlatılır. Bu yöntem, kilitlenmiş süreçleri tespit etmek ve programın akışını devam ettirmek için çok önemlidir. Hem `get()` hem de `put()` için kullanılabilir.

Bloklama Yönetimi ve timeout Parametresi

```
import multiprocessing
import queue # queue.Empty hatası için gerekli
import time

def worker(q):
    try:
        # 3 saniye veri bekle, gelmezse hata ver
        data = q.get(timeout=3)
        print(f"Worker veriyi aldı: {data}")
    except queue.Empty:
        print("Kuyruk boş! Süre doldu, worker kapanıyor.")

if __name__ == "__main__":
    q = multiprocessing.Queue()
    p = multiprocessing.Process(target=worker, args=(q,))
    p.start()

    # Ana process hiçbir şey göndermiyor...
    # Worker 3 saniye bekleyip kapanacak.
    p.join()
```

JoinableQueue Nedir?

- Standart Queue sadece veri taşır, işin tamamlanıp tamamlanmadığını takip edemez. Ancak JoinableQueue, iş takibi de yapabilen gelişmiş bir kuyruktur.
- `q.task_done()`: Tüketici (Consumer), kuyruktan aldığı bir veriyi işleyip bitirdiğinde bu metodu çağırarak Benim işim bitti sinyali verir.
- `q.join()`: Üretici (Producer), kuyruğa attığı tüm işlerin bitip bitmediğini kontrol etmek için bu metodu kullanır. `task_done()` sayısı, `put()` sayısına eşit olana kadar `join()` metodu programı bekletir.

JoinableQueue Çalışma Mantığı

- Mekanizma bir sayaç gibi çalışır:
 1. Her put() işlemi, içerdeki sayacı +1 artırır.
 2. Her task_done() işlemi, sayacı -1 azaltır.
 3. join() metodu, sayaç 0 olana kadar ana programı bloklar. Bu yapı, özellikle "Master-Worker" (Usta-Çırak) modelinde, tüm işlerin bittiğinden emin olduktan sonra programı kapatmak için kullanılır.

JoinableQueue Çalışma Mantığı

```
import multiprocessing
import time

def consumer(q):
    while True:
        item = q.get()
        print(f"İşleniyor: {item}")
        time.sleep(1) # İşlem simülasyonu
        # İşin bittiğini sisteme bildiriyoruz
        q.task_done()

if __name__ == "__main__":
    # JoinableQueue tanımlıyoruz
    jq = multiprocessing.JoinableQueue()

    # Consumer process başlatılıyor (Daemon olarak)
    p = multiprocessing.Process(target=consumer, args=(jq,))
    p.daemon = True # Ana program bitince bu da bitsin
    p.start()

    # Producer (Ana process) işleri kuyruğa ekliyor
    tasks = ["Dosya1", "Dosya2", "Dosya3"]

    for task in tasks:
        jq.put(task)
        print(f"Kuyruğa eklendi: {task}")

    print("Tüm işlerin bitmesi bekleniyor...")
    # Kuyruktaki tüm işler için task_done() gelene kadar bekle
    jq.join()

    print("Tüm işler tamamlandı! Program kapanıyor.")
```

Sonsuz Döngü Sorunu

- Consumer process'ler genellikle `while True`: döngüsü içinde sürekli veri beklerler. İşler bittiğinde `join()` ile ana programı bekletsek bile, Consumer process hala veri beklemektedir. Bu process'in kibarca sonlandırılması gerekir. Process'i zorla öldürmek (`terminate()`) veri kaybına yol açabilir.

Çözüm: Poison Pill (Zehirli Hap) Tekniği

- Bu, paralel programlamada standart bir tasarım deseni.
- Producer, işlerin bittiğini belirtmek için kuyruğa özel bir durdurma sinyali (örneğin None değeri veya "STOP" stringi) atar.
- Consumer, kuyruktan gelen veriyi kontrol eder. Eğer veri bu özel sinyalse (if data is None:), döngüyü break ile kırar ve fonksiyonu sonlandırır.

Çözüm: Poison Pill (Zehirli Hap) Tekniği

```
def worker(q):
    while True:
        data = q.get()
        # Zehirli Hap kontrolü
        if data is None:
            print("Durdurma sinyali alındı. Çıkılıyor...")
            break
        print(f"Veri işlendi: {data**2}")
if __name__ == "__main__":
    q = multiprocessing.Queue()
    p = multiprocessing.Process(target=worker, args=(q,))
    p.start()
    # İşler gönderiliyor
    q.put(10)
    q.put(20)
    # İşler bitti, Poison Pill gönderiliyor
    q.put(None)
    p.join()
```

Pipes (Borular) ile Hızlı İletişim

- multiprocessing.Pipe, iki process arasında doğrudan bağlantı kurar. Queue'ya göre daha hafiftir ve daha hızlıdır çünkü arka planda daha az senkronizasyon mekanizması çalıştırır. Pipe oluşturulduğunda iki uç (connection object) bulunur: parent_conn ve child_conn. Bu iki uç, bir telefon hattının iki ucu gibidir.

Duplex vs Simplex Bağlantı

- Pipe tanımlanırken davranış şekli belirlenebilir:
- Full-Duplex (Varsayılan): Pipe(duplex=True). Her iki uçtan da hem veri gönderilebilir hem de okunabilir. Telefon görüşmesi gibidir.
- Simplex(Half-Duplex): Pipe(duplex=False). Bir uç sadece gönderici, diğer uç sadece alıcıdır. Radyo yayını veya telsiz gibidir. Veri akış yönünü kısıtlamak bazen hataları önlemek için tercih edilir.

Pipes(Borular) ile İletişim

- Pipes yapısının kapasitesi belirlenemez.
- Bir Producer(Sağlayıcı - Gönderici) send() ile pipea bir mesaj ekler ve eklenen bu mesaj Consumer(Alıcı) tarafından recv() ile okunur.
- Birden çok Producer ve birden çok Consumer olamaz.

Pipes(Borular) ile İletişim



PIPES

Pipe Metotları: send, recv ve poll

- `send(obj)`: Veriyi boruya yazar.
- `recv()`: Veriyi okur. Eğer veri yoksa, veri gelene kadar bloklar.
- `poll()`: Boru içinde okunacak veri var mı? sorusunu sorar. True veya False döner. Bloklanmayı önlemek için, `recv()` çağırmadan önce `poll()` ile kontrol etmek iyi bir yöntemdir.

Kod Örneği – Pipe ile Ping-Pong

```
from multiprocessing import Process, Pipe

def player(conn, name):
    msg = conn.recv() # Mesaj bekle
    print(f"{name} aldı: {msg}")
    conn.send("Pong") # Cevap ver
    conn.close()

if __name__ == "__main__":
    parent_conn, child_conn = Pipe()
    p = Process(target=player, args=(child_conn, "Oyuncu B"))
    p.start()

    parent_conn.send("Ping")
    response = parent_conn.recv()
    print(f"Ana Process cevabı aldı: {response}")
    p.join()
```

Kod Örneği – Pipe ile Ping-Pong

```
from multiprocessing import Pipe, Process
import time

def ping(pipe_conn):
    while(True):
        pipe_conn.send(["Ping", time.time()])
        pong = pipe_conn.recv()
        print(pong)
        time.sleep(1)

def pong(pipe_conn):
    while True:
        ping = pipe_conn.recv()
        print(ping)
        time.sleep(1)
        pipe_conn.send(["Pong", time.time()])

if __name__ == "__main__":
    pipe_end_a, pipe_end_b = Pipe()
    Process(target=ping, args=(pipe_end_a, )).start()
    Process(target=pong, args=(pipe_end_b, )).start()
```